

AUTHENTICATION ENGINEERING

6-Day / 6-Module Master Roadmap

Dates: 24–29 August 2026

Goal: Build deep understanding of modern authentication + implement a production-oriented reusable Authentication Service.

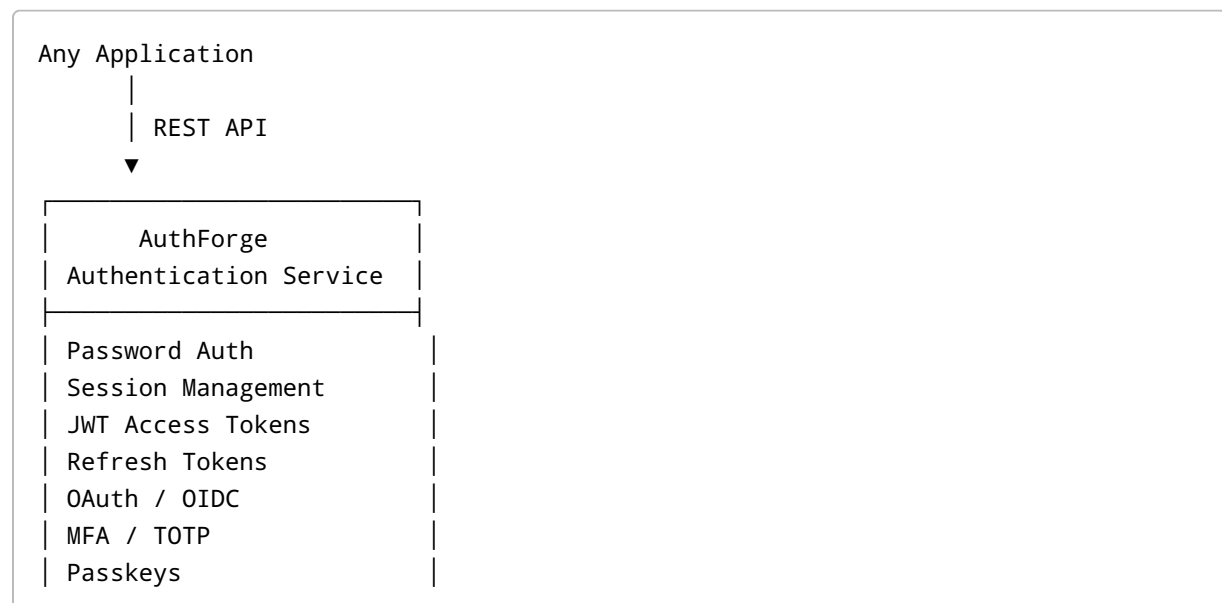
Final Project

Project Name: AuthForge — Production-Ready Authentication Service

Stack

- TypeScript
- Node.js
- Fastify
- PostgreSQL
- Prisma
- Redis
- Docker
- REST APIs
- JWT
- OAuth 2.0 / OIDC
- TOTP / MFA
- WebAuthn / Passkeys

Final capability





MODULE 1 — AUTHENTICATION FOUNDATIONS

Day 1 — Identity → Passwords → Sessions → Cookies

1. Authentication Fundamentals

- [] What is Identity?
- [] What is Authentication?
- [] What is Authorization?
- [] Authentication vs Authorization
- [] Identification vs Authentication
- [] Authentication vs Accounting
- [] Trust boundaries
- [] Client vs Server responsibilities
- [] Why HTTP is stateless
- [] How a server recognizes a returning user

2. Password Authentication

- [] Username/password authentication
- [] Password transmission
- [] Password storage
- [] Hashing
- [] Encryption
- [] Encoding
- [] Hashing vs encryption vs encoding
- [] One-way functions
- [] Salt
- [] Pepper
- [] Rainbow tables
- [] Brute-force attacks
- [] Dictionary attacks
- [] Credential stuffing
- [] Password spraying

3. Password Hashing Algorithms

- ☐ MD5
- ☐ SHA-1
- ☐ SHA-256
- ☐ Why general-purpose hashes are unsuitable for passwords
- ☐ bcrypt
- ☐ scrypt
- ☐ Argon2
- ☐ Work factor
- ☐ Memory hardness
- ☐ Password hash verification

4. HTTP Authentication

- ☐ HTTP request/response
- ☐ Headers
- ☐ Authorization header
- ☐ Cookies
- ☐ Set-Cookie
- ☐ Cookie lifecycle
- ☐ Session cookies
- ☐ Persistent cookies
- ☐ Cookie expiration

5. Cookie Security

- ☐ HttpOnly
- ☐ Secure
- ☐ SameSite
- ☐ SameSite=Lax
- ☐ SameSite=Strict
- ☐ SameSite=None
- ☐ Domain
- ☐ Path
- ☐ Cookie prefixes

6. Session Authentication

- ☐ Session ID
- ☐ Session creation
- ☐ Session lookup
- ☐ Session expiration
- ☐ Logout
- ☐ Session invalidation
- ☐ Session rotation
- ☐ Session fixation
- ☐ Session hijacking

7. Session Storage

- ☐ In-memory sessions
- ☐ Database sessions
- ☐ Redis sessions
- ☐ Stateful authentication
- ☐ Stateless authentication
- ☐ Horizontal scaling
- ☐ Sticky sessions
- ☐ Distributed sessions

Day 1 Practical

Build:

```
POST /auth/register
POST /auth/login
GET /auth/me
POST /auth/logout
```

Implement:

- ☐ User table
- ☐ Password hashing
- ☐ Login verification
- ☐ Session generation
- ☐ Session storage
- ☐ HttpOnly cookie
- ☐ Session expiration
- ☐ Logout
- ☐ Protected route

Day 1 Interview

- ☐ Authentication vs authorization?
- ☐ Why hash passwords?
- ☐ Why salt passwords?
- ☐ Why not SHA-256 for passwords?
- ☐ bcrypt vs Argon2?
- ☐ JWT vs session?
- ☐ Cookie vs localStorage?
- ☐ How does a session work?
- ☐ How do sessions scale?

MODULE 1 DONE WHEN

You can explain:

```
Browser
↓
POST /login
↓
Server
↓
Verify password hash
↓
Create session
↓
Store session
↓
Set HttpOnly cookie
↓
Browser sends cookie
↓
Server finds session
↓
User authenticated
```

MODULE 2 — JWT + TOKEN ARCHITECTURE

Day 2 — JWT → Access Tokens → Refresh Tokens

1. JWT Fundamentals

- [] What is JWT?
- [] Why JWT exists
- [] JWT structure
- [] Header
- [] Payload
- [] Signature
- [] Base64URL
- [] JWT is not encryption
- [] JWS
- [] JWE
- [] Claims

2. JWT Claims

- [] iss
- [] sub
- [] aud
- [] exp

- ☐ nbf
- ☐ iat
- ☐ jti
- ☐ Custom claims
- ☐ Registered claims
- ☐ Public claims
- ☐ Private claims

3. JWT Cryptography

- ☐ Symmetric cryptography
- ☐ HS256
- ☐ Secret keys
- ☐ Asymmetric cryptography
- ☐ RS256
- ☐ ES256
- ☐ EdDSA
- ☐ Private keys
- ☐ Public keys
- ☐ Signing
- ☐ Verification

4. JWT Validation

- ☐ Signature verification
- ☐ Algorithm validation
- ☐ Expiration validation
- ☐ Issuer validation
- ☐ Audience validation
- ☐ Subject validation
- ☐ Required claims
- ☐ Token type validation

5. Access Tokens

- ☐ Bearer tokens
- ☐ Authorization header
- ☐ Short-lived tokens
- ☐ Token expiration
- ☐ Token replay
- ☐ Token leakage

6. Refresh Tokens

- ☐ Why refresh tokens exist
- ☐ Access-token lifetime
- ☐ Refresh-token lifetime
- ☐ Refresh endpoint
- ☐ Refresh-token rotation

- ☐ Refresh-token revocation
- ☐ Refresh-token reuse detection
- ☐ Token families
- ☐ Device sessions

7. JWT Key Management

- ☐ Secret management
- ☐ Private key management
- ☐ Public key distribution
- ☐ JWKS
- ☐ Key IDs
- ☐ Key rotation
- ☐ Old-key compatibility
- ☐ Zero-downtime rotation

Day 2 Practical

Replace your Day-1 session authentication with:

```
POST /auth/login
    ↓
Access Token
    +
Refresh Token
```

Implement:

- ☐ JWT creation
- ☐ JWT verification
- ☐ Access token
- ☐ Refresh token
- ☐ Token expiration
- ☐ Refresh endpoint
- ☐ Refresh rotation
- ☐ Refresh-token storage
- ☐ Logout
- ☐ Logout all devices
- ☐ Token revocation
- ☐ RS256
- ☐ JWKS endpoint

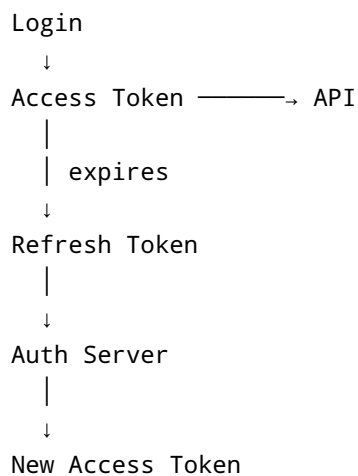
Day 2 Interview

- ☐ What exactly is inside a JWT?
- ☐ Is JWT encrypted?
- ☐ JWT vs session?

- ☐ HS256 vs RS256?
- ☐ Why asymmetric signing?
- ☐ What is JWKS?
- ☐ How do you revoke a JWT?
- ☐ Why short-lived access tokens?
- ☐ Why refresh tokens?
- ☐ How does refresh-token rotation work?

MODULE 2 DONE WHEN

You can explain:



MODULE 3 — WEB SECURITY + AUTH ATTACKS

Day 3 — Break Your Own Authentication System

This module is intentionally attack-focused.

1. CSRF

- ☐ What is CSRF?
- ☐ Why CSRF exists
- ☐ Cookie-based authentication
- ☐ CSRF attack flow
- ☐ CSRF tokens
- ☐ Synchronizer token pattern
- ☐ Double-submit cookie
- ☐ SameSite protection
- ☐ Origin validation

- ☐ Referer validation

2. XSS

- ☐ Reflected XSS
- ☐ Stored XSS
- ☐ DOM XSS
- ☐ XSS → token theft
- ☐ XSS → authenticated actions
- ☐ HttpOnly protection
- ☐ CSP
- ☐ Output encoding
- ☐ Sanitization

3. Session Attacks

- ☐ Session fixation
- ☐ Session hijacking
- ☐ Session replay
- ☐ Session prediction
- ☐ Session theft
- ☐ Session rotation

4. JWT Attacks

- ☐ JWT tampering
- ☐ Signature bypass
- ☐ Algorithm confusion
- ☐ algorithm issue
- ☐ Weak secrets
- ☐ Key confusion
- ☐ Token replay
- ☐ Expired-token attacks
- ☐ Claim manipulation
- ☐ Audience confusion
- ☐ Issuer confusion

5. Credential Attacks

- ☐ Brute force
- ☐ Credential stuffing
- ☐ Password spraying
- ☐ Account enumeration
- ☐ Timing attacks
- ☐ Login rate limiting

6. Token Security

- ☐ Token leakage

- ☐ Token storage
- ☐ Access-token theft
- ☐ Refresh-token theft
- ☐ Refresh-token reuse
- ☐ Token revocation
- ☐ Token rotation

7. Rate Limiting

- ☐ IP-based rate limiting
- ☐ User-based rate limiting
- ☐ Device-based rate limiting
- ☐ Distributed rate limiting
- ☐ Redis rate limiter
- ☐ Progressive delays
- ☐ Account lockout
- ☐ CAPTCHA

Day 3 Practical — Attack Lab

Attack your own AuthForge.

- ☐ Modify JWT payload
- ☐ Modify JWT signature
- ☐ Send expired JWT
- ☐ Send wrong audience
- ☐ Send wrong issuer
- ☐ Try algorithm manipulation
- ☐ Replay refresh token
- ☐ Steal session cookie
- ☐ Attempt CSRF
- ☐ Attempt XSS
- ☐ Brute-force login
- ☐ Enumerate accounts
- ☐ Abuse password reset
- ☐ Abuse OTP endpoints

Then:

- ☐ Fix every vulnerability
- ☐ Add rate limiting
- ☐ Add security headers
- ☐ Add audit logs
- ☐ Add authentication events

Day 3 Interview

- ☐ CSRF vs XSS?

- ☐ Can HttpOnly prevent XSS?
- ☐ Why is localStorage risky?
- ☐ Why does JWT in Authorization header change CSRF risk?
- ☐ How does session fixation happen?
- ☐ How do you revoke stolen refresh tokens?
- ☐ How do you defend login against brute force?
- ☐ How do you prevent account enumeration?

MODULE 3 DONE WHEN

You can answer:

"Assume an attacker has stolen a user's token. What can they do, how long can they use it, and how do you stop them?"

MODULE 4 — OAUTH 2.0 + OIDC + SOCIAL LOGIN

Day 4 — Delegated Authentication & Identity

1. OAuth 2.0 Fundamentals

- ☐ Why OAuth exists
- ☐ Resource Owner
- ☐ Client
- ☐ Authorization Server
- ☐ Resource Server
- ☐ Access Token
- ☐ Authorization Code
- ☐ Redirect URI
- ☐ Scope
- ☐ Consent

2. OAuth Flows

- ☐ Authorization Code Flow
- ☐ Authorization Code + PKCE
- ☐ Client Credentials
- ☐ Device Authorization
- ☐ Refresh Token Grant
- ☐ Understand implicit flow
- ☐ Why implicit flow is discouraged

3. PKCE

- ☐ code_verifier
- ☐ code_challenge

- ☐ S256
- ☐ Authorization request
- ☐ Authorization response
- ☐ Token exchange
- ☐ Authorization-code interception
- ☐ Authorization-code injection

4. OAuth Security

- ☐ state
- ☐ nonce
- ☐ Redirect URI validation
- ☐ Open redirects
- ☐ Token leakage
- ☐ Authorization-code theft
- ☐ Authorization-code injection
- ☐ Mix-up attacks
- ☐ Token replay

5. OIDC

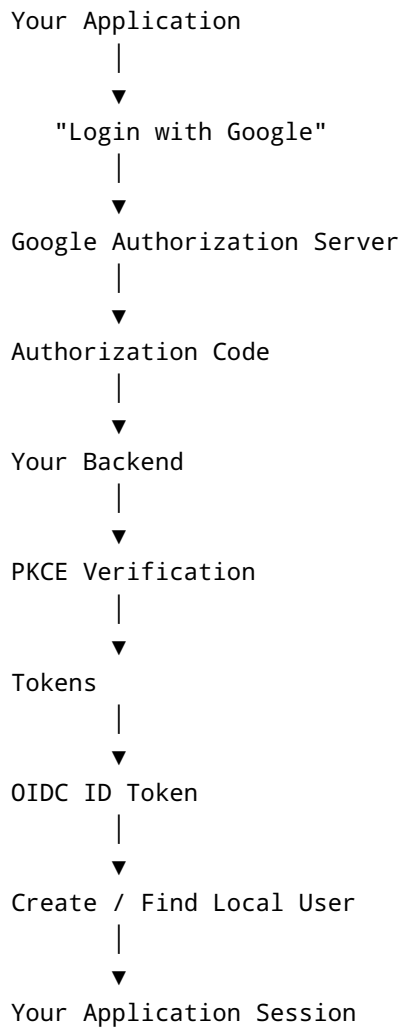
- ☐ Why OIDC exists
- ☐ OAuth vs OIDC
- ☐ Identity layer
- ☐ ID Token
- ☐ Access Token
- ☐ UserInfo endpoint
- ☐ Issuer
- ☐ Client ID
- ☐ Client Secret
- ☐ Discovery
- ☐ JWKS
- ☐ Nonce

6. Social Login

- ☐ Google Login
- ☐ GitHub Login
- ☐ Microsoft Login
- ☐ Apple Login
- ☐ Federated identity
- ☐ Identity providers
- ☐ Account linking
- ☐ Multiple providers
- ☐ Provider subject IDs
- ☐ Email collision

Day 4 Practical

Implement:



- ☐ OAuth client
- ☐ Authorization Code
- ☐ PKCE
- ☐ state
- ☐ nonce
- ☐ ID token validation
- ☐ JWKS validation
- ☐ Google login
- ☐ Account linking

Day 4 Interview

- ☐ OAuth vs OIDC?

- ☐ OAuth vs JWT?
- ☐ Why OAuth isn't authentication?
- ☐ What is PKCE?
- ☐ Why state?
- ☐ Why nonce?
- ☐ Access token vs ID token?
- ☐ Authorization server vs resource server?
- ☐ Explain Google login from beginning to end.

MODULE 4 DONE WHEN

You can draw the complete Google login flow on a whiteboard **without looking at notes**.

MODULE 5 — MFA + PASSKEYS + AUTHORIZATION

Day 5 — Modern Identity Security

1. MFA

- ☐ Authentication factors
- ☐ Something you know
- ☐ Something you have
- ☐ Something you are
- ☐ MFA vs 2FA
- ☐ TOTP
- ☐ HOTP
- ☐ Authenticator apps
- ☐ SMS OTP
- ☐ Email OTP
- ☐ Recovery codes
- ☐ MFA enrollment
- ☐ MFA verification
- ☐ MFA reset
- ☐ Step-up authentication

2. MFA Security

- ☐ OTP expiration
- ☐ OTP replay
- ☐ OTP brute force
- ☐ Rate limiting
- ☐ SIM swapping
- ☐ MFA fatigue
- ☐ Recovery-code security

3. Passkeys / WebAuthn

- ☐ Passwordless authentication
- ☐ Passkeys
- ☐ WebAuthn
- ☐ FIDO2
- ☐ Public-key credentials
- ☐ Relying Party
- ☐ Authenticator
- ☐ Challenge
- ☐ Credential ID
- ☐ Public key
- ☐ Private key
- ☐ Registration ceremony
- ☐ Authentication ceremony
- ☐ Origin binding
- ☐ User verification
- ☐ Platform authenticators
- ☐ Roaming authenticators

4. Authorization

- ☐ Authentication vs authorization
- ☐ Roles
- ☐ Permissions
- ☐ RBAC
- ☐ ABAC
- ☐ Resource ownership
- ☐ Resource-level authorization
- ☐ Permission inheritance
- ☐ Multi-tenant authorization
- ☐ Tenant isolation
- ☐ Policy-based authorization

5. Account Lifecycle

- ☐ Registration
- ☐ Email verification
- ☐ Login
- ☐ Logout
- ☐ Password change
- ☐ Password reset
- ☐ Email change
- ☐ MFA enrollment
- ☐ MFA removal
- ☐ Account recovery
- ☐ Device management
- ☐ Active sessions

- ☐ Re-authentication
- ☐ Account deletion

Day 5 Practical

Implement:

```

Password
  ↓
MFA Challenge
  ↓
TOTP
  ↓
Authenticated
  
```

Then:

```

User
  ↓
Role
  ↓
Permissions
  ↓
Resource
  ↓
Authorization Decision
  
```

And implement:

- ☐ TOTP enrollment
- ☐ QR code
- ☐ TOTP verification
- ☐ Recovery codes
- ☐ MFA challenge
- ☐ RBAC
- ☐ Permissions
- ☐ Resource ownership
- ☐ Tenant isolation

Day 5 Interview

- ☐ How does TOTP work?
- ☐ How do authenticator apps generate codes?
- ☐ Why is SMS MFA weaker?
- ☐ How do passkeys work?

- [] Where is the private key?
- [] Why are passkeys phishing resistant?
- [] RBAC vs ABAC?
- [] How would you design authorization for a SaaS?

MODULE 5 DONE WHEN

You can explain why:

```
Password
  ↓
Password + TOTP
  ↓
Passkey
```

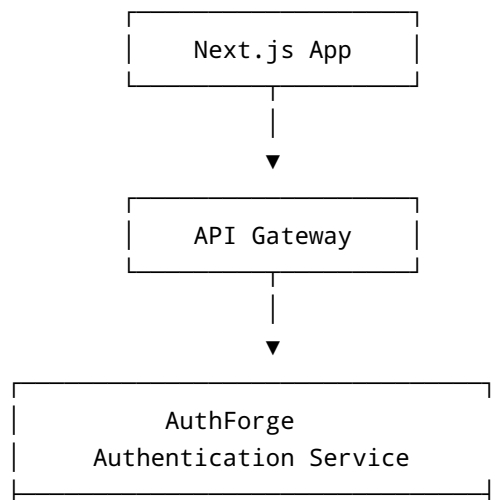
represents an evolution in authentication security.

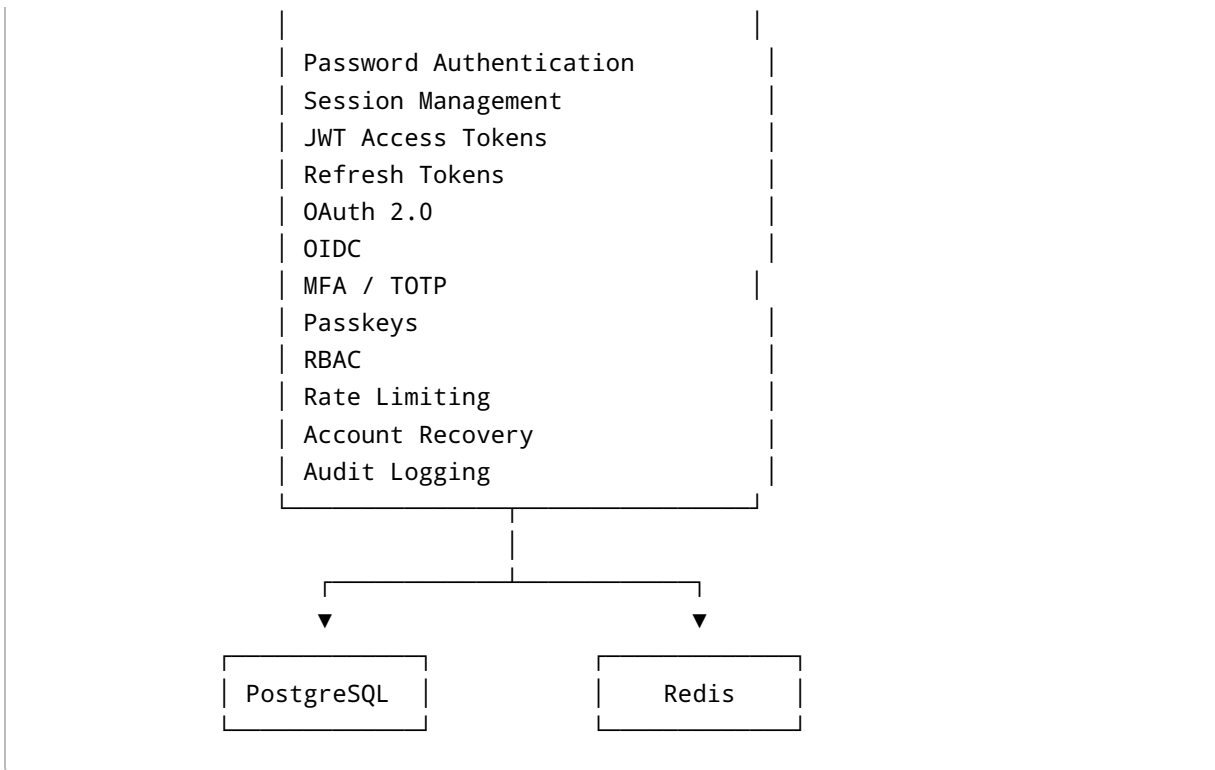
MODULE 6 — PRODUCTION AUTH SERVICE + SYSTEM DESIGN

Day 6 — Turn Everything Into a Real Product

This is where the six days become one **resume-worthy project**.

1. AuthForge Architecture





2. Database Design

Create:

- [] users
- [] credentials
- [] sessions
- [] refresh_tokens
- [] oauth_accounts
- [] roles
- [] permissions
- [] role_permissions
- [] user_roles
- [] mfa_methods
- [] recovery_codes
- [] passkeys
- [] password_reset_tokens
- [] email_verification_tokens
- [] audit_logs
- [] devices

Understand every relationship.

3. API Design

Authentication

- [] POST /auth/register
- [] POST /auth/login
- [] POST /auth/logout
- [] POST /auth/refresh
- [] GET /auth/me
- [] POST /auth/logout-all

Password

- [] POST /auth/password/forgot
- [] POST /auth/password/reset
- [] POST /auth/password/change

Email

- [] POST /auth/email/verify
- [] POST /auth/email/resend

OAuth

- [] GET /auth/oauth/:provider
- [] GET /auth/oauth/:provider/callback

MFA

- [] POST /auth/mfa/enroll
- [] POST /auth/mfa/verify
- [] POST /auth/mfa/disable

Sessions

- [] GET /auth/sessions
- [] DELETE /auth/sessions/:id
- [] DELETE /auth/sessions

Authorization

- [] GET /auth/permissions
- [] GET /auth/roles

Passkeys

- [] POST /auth/passkeys/register/options
- [] POST /auth/passkeys/register/verify
- [] POST /auth/passkeys/login/options

- ☐ POST /auth/passkeys/login/verify
-

4. Production Security

- ☐ Password hashing
 - ☐ Secure cookies
 - ☐ HttpOnly cookies
 - ☐ SameSite
 - ☐ CSRF protection
 - ☐ XSS protection
 - ☐ Rate limiting
 - ☐ Brute-force protection
 - ☐ Account enumeration protection
 - ☐ Token rotation
 - ☐ Refresh-token reuse detection
 - ☐ Token revocation
 - ☐ Key rotation
 - ☐ Secret management
 - ☐ Audit logs
 - ☐ Security headers
 - ☐ Input validation
 - ☐ Request validation
 - ☐ Error handling
 - ☐ Secure logging
 - ☐ No sensitive data in logs
-

5. Distributed Authentication

- ☐ Stateless JWT verification
- ☐ Redis session storage
- ☐ Distributed rate limiting
- ☐ API Gateway
- ☐ Service-to-service authentication
- ☐ JWT propagation
- ☐ Service identity
- ☐ mTLS awareness
- ☐ Key distribution
- ☐ JWKS
- ☐ Key rotation
- ☐ Clock skew
- ☐ Token expiration
- ☐ Global logout
- ☐ Distributed revocation

6. Testing

Unit Tests

- ☐ Password hashing
- ☐ Password verification
- ☐ JWT generation
- ☐ JWT verification
- ☐ Token expiration
- ☐ Permission checks

Integration Tests

- ☐ Registration
- ☐ Login
- ☐ Logout
- ☐ Refresh
- ☐ Password reset
- ☐ OAuth
- ☐ MFA
- ☐ Session management

Security Tests

- ☐ JWT tampering
- ☐ Expired JWT
- ☐ Invalid signature
- ☐ Wrong issuer
- ☐ Wrong audience
- ☐ Refresh-token replay
- ☐ CSRF
- ☐ XSS
- ☐ Brute force
- ☐ Rate limiting
- ☐ Authorization bypass
- ☐ Tenant isolation

7. Deployment

- ☐ Dockerize AuthForge
- ☐ PostgreSQL container
- ☐ Redis container
- ☐ Environment configuration
- ☐ Secret management

- ☐ Production configuration
 - ☐ Health endpoint
 - ☐ Readiness endpoint
 - ☐ Logging
 - ☐ Monitoring
 - ☐ CI/CD
 - ☐ Database migrations
 - ☐ HTTPS
 - ☐ Reverse proxy
 - ☐ Production deployment
-

8. Developer Experience

The service should be usable by another application.

Create:

- ☐ API documentation
 - ☐ OpenAPI specification
 - ☐ Postman collection
 - ☐ Authentication integration guide
 - ☐ Example Next.js client
 - ☐ Example Fastify client
 - ☐ Docker Compose
 - ☐ `.env.example`
 - ☐ Database migration scripts
 - ☐ Architecture diagram
 - ☐ Authentication flow diagrams
-

9. Final Resume Project

AuthForge

Position it as:

AuthForge — Production-Ready Authentication & Identity Service

Features

- ☐ Password authentication
- ☐ Session-based authentication
- ☐ JWT access/refresh tokens
- ☐ Refresh-token rotation

- ☐ OAuth 2.0
 - ☐ OpenID Connect
 - ☐ Social login
 - ☐ MFA / TOTP
 - ☐ Passkeys / WebAuthn
 - ☐ RBAC
 - ☐ Multi-tenant authorization
 - ☐ Redis-backed sessions
 - ☐ Distributed rate limiting
 - ☐ Token revocation
 - ☐ JWT key rotation
 - ☐ Audit logging
 - ☐ Dockerized deployment
 - ☐ OpenAPI documentation
 - ☐ Automated security tests
-

10. FINAL INTERVIEW CHECKPOINT

Before declaring the project complete:

Level 1 — Fundamentals

- ☐ Explain authentication
- ☐ Explain authorization
- ☐ Explain password hashing
- ☐ Explain sessions
- ☐ Explain cookies

Level 2 — Tokens

- ☐ Explain JWT
- ☐ Explain JWT signature
- ☐ Explain HS256
- ☐ Explain RS256
- ☐ Explain access tokens
- ☐ Explain refresh tokens
- ☐ Explain token rotation
- ☐ Explain token revocation

Level 3 — Web Security

- ☐ Explain CSRF
- ☐ Explain XSS
- ☐ Explain session fixation
- ☐ Explain session hijacking
- ☐ Explain JWT attacks

- ☐ Explain credential stuffing
- ☐ Explain rate limiting

Level 4 — Federation

- ☐ Explain OAuth
- ☐ Explain Authorization Code
- ☐ Explain PKCE
- ☐ Explain state
- ☐ Explain nonce
- ☐ Explain OIDC
- ☐ Explain ID tokens
- ☐ Explain Google Login

Level 5 — Modern Authentication

- ☐ Explain MFA
- ☐ Explain TOTP
- ☐ Explain recovery codes
- ☐ Explain passkeys
- ☐ Explain WebAuthn
- ☐ Explain RBAC
- ☐ Explain ABAC

Level 6 — System Design

- ☐ Design Auth Service
- ☐ Design authentication for 1M users
- ☐ Design authentication for 100M users
- ☐ Design SSO
- ☐ Design multi-device sessions
- ☐ Design global logout
- ☐ Design token revocation
- ☐ Design microservice authentication
- ☐ Design key rotation
- ☐ Design multi-tenant authorization

FINAL DEFINITION OF DONE

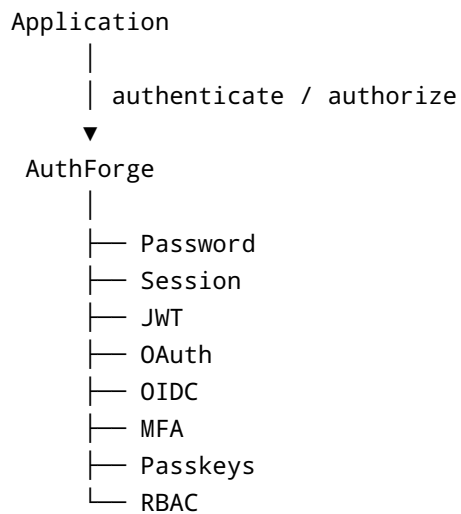
At the end of Day 6, you should have:

- ☐ Deep theoretical understanding
- ☐ Working authentication service
- ☐ PostgreSQL schema
- ☐ Redis integration
- ☐ JWT implementation

- [] Refresh-token architecture
- [] OAuth/OIDC
- [] Social login
- [] MFA
- [] Passkeys
- [] RBAC
- [] Security protections
- [] Automated tests
- [] Docker setup
- [] API documentation
- [] Architecture diagram
- [] GitHub repository
- [] Resume-ready project description

The final standard

You should be able to take **any existing application**, point its authentication layer toward AuthForge, and say:



And, more importantly, you should be able to explain **why every component exists, what problem it solves, what security problem it introduces, how it is implemented, and how it scales.**